

SYMBOL TABLE

A Complete Study Note

Topics Covered

Scoping Rules • Symbol Table Design • List of Hashtables • Hashtable of Lists • Running-Time Analysis • Worked Examples • Type Checking

Prepared by

Azmari Sultana

Table of Contents

1. Introduction.....	3
1.1 Why We Need a Symbol Table.....	3
1.2 Two-Phase Static Checking.....	3
1.3 What Is a Symbol Table?.....	3
1.4 The Two Questions a Symbol Table Must Answer.....	3
2. Scoping Rules.....	4
2.1 Reusing Names for Different Kinds of Entities.....	4
2.2 Java vs. C++: Shadowing Rules.....	4
2.3 Static Scoping.....	5
2.4 Dynamic Scoping.....	6
2.5 Using a Name Before Its Declaration.....	7
2.6 Assumptions Used for the Rest of This Note.....	7
3. Operations a Symbol Table Must Support.....	8
4. Method 1 — List of Hashtables.....	8
4.1 The Core Idea.....	8
4.2 Worked Example A.....	8
4.3 Worked Example B — Function Call and Locals.....	8
4.4 Why Method/Function Names Go in the Outermost Table.....	9
4.5 The Four Core Operations.....	10
4.6 Running-Time Analysis.....	10
4.7 Scoping Example.....	10
5. Method 2 — Hashtable of Lists.....	10
5.1 The Core Idea.....	10
5.2 Worked Example.....	10
5.3 The Four Core Operations.....	11
5.4 Running-Time Analysis.....	11
5.5 Test Yourself — Full Trace.....	11
6. Comparing the Two Implementations.....	12
7. Type Checking.....	12
7.1 Kinds of Type Errors Beyond Operators.....	12

1. Introduction

1.1 Why We Need a Symbol Table

The parser only checks that a program follows the correct syntax (grammar) of the language. Some errors are structurally valid but still incorrect:

- A variable should not be declared more than once in the same scope.
- A variable should not be used before it is declared.
- The type on the left-hand side of an assignment should match the type on the right-hand side.
- A method should be called with the correct number and types of arguments.

These are caught by the static semantic analyzer, the compiler phase that runs after parsing, by walking the abstract-syntax tree (AST).

1.2 Two-Phase Static Checking

Phase 1 — Build Symbol Table & Resolve Names: For every scope, process each declaration (add entry, report duplicates) and each use (find matching declaration, report if undeclared, link the AST ID node to the symbol-table entry).

Phase 2 — Type Checking: Walk the AST again using the symbol-table information (types) to determine the type of every expression and report type errors.

1.3 What Is a Symbol Table?

The symbol table is a compiler data structure that tracks every name declared in a program — classes, fields, methods, and variables. Each entry associates a name with attributes:

- Which kind of name it is (class, field, method, variable)
- Its type
- Its nesting level (scope depth)
- Its runtime location (memory offset / register)

1.4 The Two Questions a Symbol Table Must Answer

- Given a declaration of a name, is there already a declaration of the same name in the current scope (multiply declared)?
- Given a use of a name, which declaration does it correspond to (using the "most closely nested" rule), or is it undeclared?

Once every declaration has been processed and every use linked (by pointer) to its symbol-table entry, the symbol table is no longer needed, since no further name-based lookups occur.

Q: What is a symbol table in the context of a compiler? Explain the different implementations of a symbol table. Discuss the advantages and disadvantages of each implementation.

Ans: A symbol table is a compiler data structure that records every declared name (class, field, method, variable) together with attributes such as kind, type, nesting level, and runtime location, and is used to detect multiply-declared and undeclared names and to link each use to its declaration. Two common implementations:

List of Hashtables: one hashtable per open scope, linked in a list (current scope at front). Advantage: $O(1)$ scope entry/exit-like cost with $O(1)$ scope exit and $O(1)$ declaration. Disadvantage: a use may need $O(\text{depth of nesting})$ lookups across multiple tables.

Hashtable of Lists: one shared hashtable for the whole program; each name maps to a list of declarations ordered innermost-first. Advantage: $O(1)$ lookup for a use regardless of nesting depth. Disadvantage: scope exit costs $O(\text{number of names in the table})$, since every entry must be checked.

Q: Explain how symbol tables are used to determine what variable comes from what memory location during translating expressions and statements.

Ans: Each symbol-table entry stores, along with the name and type, the variable's runtime location (e.g., a stack offset or register). When the code generator translates an expression or statement, it looks up the identifier's entry — already linked to the corresponding AST node during Phase 1 — and uses the stored location attribute to emit the correct load/store instruction for that variable.

Q: Explain how symbol tables are used to manage scope and detect redeclaration or undefined variable errors during semantic analysis.

Ans: On scope entry, a new table (List of Hashtables) or an incremented level (Hashtable of Lists) is created to represent the new scope. Each declaration is inserted with its nesting level; if the name already exists in the current scope, a "multiply declared variable" error is reported. Each use of a name is looked up in the current scope and then outward through enclosing scopes; if it is not found anywhere, an "undeclared variable" error is reported. On scope exit, the declarations belonging to that scope are removed, restoring visibility of the enclosing scope's declarations.

Q: If a symbol table stores only the names of identifiers and omits all other associated attributes, what types of errors may arise during the compilation process?

Ans: Without stored types, type-mismatch errors (e.g., assigning an incompatible type, invalid operator operands) cannot be detected. Without nesting-level/scope information, the compiler cannot correctly distinguish a same-scope redeclaration (multiply-declared error) from a legal redeclaration in a nested scope, nor correctly resolve which declaration a use refers to. Without kind information, errors such as calling a non-method as a method, or using a name in an invalid context, cannot be caught. Without runtime-location information, correct code for accessing the variable cannot be generated.

2. Scoping Rules

2.1 Reusing Names for Different Kinds of Entities

The same name may be declared multiple times if the declarations are in different scopes and/or are for different kinds of names. Example (legal Java, not recommended):

```
class Test {
    int Test;
    void Test( ) {
        double Test; // could also be declared int
    }
}
```

Java and C++ both allow method/function overloading — the same method name can be reused if the number and/or types of parameters differ.

2.2 Java vs. C++: Shadowing Rules

Java: a variable *x* cannot be declared inside a method if a parameter *x*, or a variable *x* in an enclosing block, already exists.

C++: such shadowing is allowed — an inner declaration hides the outer one within its own scope. Legal in C++, illegal in Java:

```
void f( int k ) { // k is a parameter
```

```

int k = 0;        // also a local variable
while (...) {
    int k = 1;    // and another local variable, inside the loop
    ...
}
}

```

2.3 Static Scoping

Static scoping: the correspondence between a use of a name and its declaration is decided at compile time, based purely on how the source code is nested. C++ and Java use static scoping and the "most closely nested" rule: a use of *x* matches the declaration in the most closely enclosing scope whose declaration precedes the use.

In C++, one outermost scope holds all function and global-variable names. Each function has its own scope for parameters, one for its body, and one more for each nested block/for-loop.

Worked Example — Static Scoping

```

void f( int k ) { // k is a parameter
    int k = 0;    // also a local variable
    while (...) {
        int k = 1; // and another local variable, inside the loop
        ...
    }
}

```

A use of *k* inside the loop matches the innermost declaration (value 1); a use of *k* outside the loop matches the function-body declaration (value 0).

Q: Consider the names declared in the following code. For each, determine whether it is legal according to the rules used in Java.

```

class animal {
    // methods
    void attack(int animal) {
        for (int animal=0; animal<10; animal++) {
            int attack;
        }
    }
    int attack(int x) {
        for (int attack=0; attack<10; attack++) {
            int animal;
        }
    }
    void animal() { }
    // fields
    double attack;
    int attack;
    int animal;
}

```

Ans:

- `void attack(int animal)`: parameter `animal` is legal. The nested `for (int animal=0; ...)` is ILLEGAL — Java forbids redeclaring a name already used as a parameter/local of the enclosing method.
- `int attack;` inside that loop is legal by itself (no conflicting local exists).
- `int attack(int x)`: parameter `x` is legal. `for (int attack=0; ...)` is legal, and `int animal;` inside it is legal.
- `void attack(int animal)` and `int attack(int x)` together are ILLEGAL — Java overloading requires different parameter types; both take a single `int`.
- `void animal()` is legal — a method may share its name with the class.

- double attack; and int attack; together are ILLEGAL — a class cannot declare two fields with the same name.
- int animal; is legal by itself.

Q: Consider the following C++ code. For each use of a name, determine which declaration it corresponds to (or whether it is a use of an undeclared name).

```
int k=10, x=20;
void foo(int k) {
    int a = x;
    int x = k;
    int b = x;
    while (...) {
        int x;
        if (x == k) {
            int k, y;
            k = y = x;
        }
        if (x == k) {
            int x = y;
        }
    }
}
```

Ans:

- foo(int k) shadows the global k throughout foo's body.
- int a = x; — no local x yet, so x is the global x (20).
- int x = k; — declares local x; k on the right is foo's parameter k.
- int b = x; — matches the local x just declared.
- while (...) { int x; ... } — a new, more deeply nested x, shadowing foo's local x.
- First if (x == k) — x matches the while-loop's x; k matches foo's parameter k.
- { int k, y; k = y = x; } — new k, y declared here; x still refers to the while-loop's x.
- Second if (x == k) — the inner if-block has exited, so k matches foo's parameter k again; x still matches the while-loop's x.
- { int x = y; } — y was only declared in the FIRST if-block, which has already exited (the two ifs are siblings) — this use of y is UNDECLARED (compile error).

2.4 Dynamic Scoping

Dynamic scoping: a use of a variable with no matching declaration in the current function is resolved to the declaration in the most-recently-called still-active function, determined at run time. Lisp, APL, and Snobol use dynamic scoping. It is considered poor design because a single use can match many different declarations depending on the call sequence.

Worked Example — Dynamic Scoping

```
void main() {
    f1();
    f2();
}
void f1() {
    int x = 10;
    g();
}
void f2() {
```

```

    String x = "hello";
    f3();
    g();
}
void f3() {
    double x = 30.5;
}
void g() {
    print(x);
}

```

Output: "10 hello". main() calls f1 (x=10) which calls g() → prints 10. main() then calls f2 (x="hello"); f2 calls f3 (which returns before g() is called), then f2 calls g() → the most recently active function is f2, so it prints "hello".

Q: Assuming that dynamic scoping is used, what is output by the following program?

```

void main() {
    int x = 0;
    f1();
    g();
    f2();
}
void f1() {
    int x = 10;
    g();
}
void f2() {
    int x = 20;
    f1();
    g();
}
void g() {
    print(x);
}

```

Ans: 10 0 10 20 — Step 1: main calls f1 (x=10) → g() prints 10. Step 2: main calls g() directly, active function is main (x=0) → prints 0. Step 3: main calls f2 (x=20); f2 calls f1 (x=10) → g() prints 10. Step 4: back in f2, f2 calls g() again, active function is f2 (x=20) → prints 20.

2.5 Using a Name Before Its Declaration

In Java, a method or field name may be used before its textual declaration, but a local variable may not:

```

class Test {
    void f() {
        val = 0; // field val not yet declared -- OK
        g();     // method g not yet declared -- OK
        x = 1;   // variable x not yet declared -- ERROR!
        int x;
    }
    void g() {}
    int val;
}

```

2.6 Assumptions Used for the Rest of This Note

- Uses static scoping.
- Requires that all names be declared before they are used.
- Does not allow multiple declarations of a name in the same scope, even for different kinds of names.
- Does allow the same name to be declared in multiple nested scopes, but only once per scope.

- Uses the same scope for a method's parameters and the local variables declared at the beginning of the method.

3. Operations a Symbol Table Must Support

- Look up a name in the current scope only — to check if it is multiply declared.
- Look up a name in the current and enclosing scopes — to check for an undeclared name and to link a use to its symbol-table entry.
- Insert a new name into the symbol table with its attributes.
- Perform bookkeeping needed when a new scope is entered.
- Perform bookkeeping needed when a scope is exited.

Each symbol-table entry in this note is simplified to store only: the symbol name, its type, and the nesting level of its declaration. Two implementation strategies are studied: a list of hashtables, and a table of lists.

4. Method 1 — List of Hashtables

4.1 The Core Idea

The symbol table is a list of hashtables, one per currently visible (open) scope. The innermost scope's table sits at the front of the list; each successively enclosing scope's table follows, ending with the outermost scope's table at the back.

4.2 Worked Example A

```
void f(int a, int b) {
    double x;
    while (...) {
        int x, y;
        ...
    }
}
void g() { f(); }
```

Bucket size 7 used for illustration. State after declarations inside the while loop are processed:

Bucket	while-Body Table (Level 3) — FRONT	f's Body Table (Level 2)	Outermost Table (Level 1)
0	x: int, 3	a: int, 2	f: (int x int) -> void, 1
1	y: int, 3	b: int, 2	
2		x: double, 2	

Method name f is stored in the outermost scope's table, not in the same table as its own parameters/locals. There are two separate entries for x — one in the level-2 table (double) and one in the level-3 table (int) — because each scope has its own hashtable.

On scope exit (leaving the while loop, then f), the front table is simply removed from the list and the level number decremented.

4.3 Worked Example B — Function Call and Locals

```
int func(int a, float b) {
    return a+b;
}
void main () {
    int a, b, c, i;
    int e, f[10];
```

```

a = 1;
b = 2;
c = func(a, b);
float d;
}

```

After both function headers are processed, both names live in the same outermost table:

Bucket	Outermost-Scope Table
0	func: (int x float) -> int, 1
1	main: () -> void, 1

After main's body scope is entered, its parameters/locals fill a new front table:

Bucket	main's Body Table (Level 2) — FRONT	Outermost Table (Level 1)
0	a: int, 2	func: (int x float) -> int, 1
1	b: int, 2	main: () -> void, 1
2	c: int, 2	
4	e: int, 2	
5	f: (10), int, 2	
6	d: float, 2	

Processing "c = func(a, b);" is a use of func: lookup fails in main's body table, then succeeds in the outermost table — this is why function names must live in the outer table.

4.4 Why Method/Function Names Go in the Outermost Table

If a function's name were stored in its own body's table, that table would be discarded as soon as the function's scope is exited, and the name would be permanently lost, so no other function could ever call it. Method names are therefore always inserted into the outermost scope's table.

Q: Recall that Java does not allow the same name to be used for a local variable of a method and for another local variable declared inside a nested scope in the method body. Even with this restriction, it is not a good idea to put all of a method's local variables in the same table. Why not?

Ans: Doing so would defeat the purpose of a list of tables — making scope exit fast and automatic. Scope exit needs to discard exactly the declarations made inside the block that just ended, without disturbing the enclosing scope. If all locals shared one table, the compiler would need to scan and remove entries by nesting level manually, losing the $O(1)$ scope-exit design.

Q: C++ does not use exactly the scoping rules that we have been assuming. In particular, C++ does allow a function to have both a parameter and a local variable with the same name (any uses of the name refer to the local variable). Consider the following code. Draw the symbol table as it would be after processing the declarations in the body of f under: the scoping rules we have been assuming; C++ scoping rules

```

void g(int x, int a) { }
void f(int x, int y, int z) {
    int a, b, x;
    ...
}

```

Ans: Under this note's assumptions: int a, b, x; is ILLEGAL, because parameters and top-level locals share one scope, and x already exists as a parameter — a "multiply declared variable" error is reported for x; only a and b are inserted. Under real C++ rules: this program is legal — the parameter x and the local x live in two separate nested scopes, so the table ends up with two entries for x (outer parameter, inner local, deeper level), plus y, z (parameters) and a, b (locals). A use of x inside f's body matches the innermost (local) x.

Q: Which of the four operations described above (scope entry, process a declaration, process a use, scope exit) would we need to change to use the following rules for name reuse instead of C++ rules: the same name can be used within one scope as long as the uses are for different kinds of names, and the same name cannot be used for more than one variable declaration in a nested scope?

Ans: Scope entry and scope exit do not change. Process a declaration must change: instead of checking only "is x already in the front table", it must also check the kind of name being declared, and report "multiply declared" only when two declarations of the same kind share a name in the same scope — typically requiring entries keyed by (name, kind). Process a use must also change: since a name can now denote more than one kind of entity, the lookup must use the context of the use to select the correct entry.

4.5 The Four Core Operations

Operation	What Happens
On scope entry	Increment the current level number; add a new empty hashtable to the front of the list.
Process a declaration of x	Look up x only in the first (frontmost) table. If found, issue a "multiply declared variable" error; otherwise insert x into that table.
Process a use of x	Look up x starting from the first table; if not found, look up in each successive table. If not found anywhere, issue an "undeclared variable" error.
On scope exit	Remove the first table from the list and decrement the current level number.

4.6 Running-Time Analysis

Operation	Running Time	Why
Scope entry	$O(\text{bucket count})$	A new, empty hashtable of fixed size must be allocated.
Process a declaration	$O(1)$ expected	Hashing gives constant expected time for one table.
Process a use	$O(\text{depth of nesting})$ worst case	Every table in the list may need to be searched.
Scope exit	$O(1)$	Removing the front table from a linked list is constant time.

4.7 Scoping Example

C++ allows a function to have both a parameter and a local variable with the same name; uses of the name refer to the local variable.

5. Method 2 — Hashtable of Lists

5.1 The Core Idea

There is one large hashtable for the whole program. Every distinct variable name with at least one currently visible declaration gets one entry, mapping to a list of symbol-table entries. The front (first) item is the most closely enclosing declaration; later items correspond to declarations in enclosing scopes. The stored nesting level lets the compiler determine whether the most closely enclosing declaration was made in the current scope or an enclosing one.

5.2 Worked Example

```
int f (int a) {
    float x, y;
    while (...) {
        char x, y, z;
    }
}
void g () {
    int x;
```

```
f(1);
}
```

After processing declarations inside the while loop:

Name	List of Declarations (innermost -> outermost)
f	int -> int, level 1
a	int, level 2
x	char, level 3 -> float, level 2
y	char, level 3 -> float, level 2
z	char, level 3

x and y each have two list entries (declared at level 2, redeclared at level 3); z has only one entry.

5.3 The Four Core Operations

Operation	What Happens
On scope entry	Increment the current level number. No new table is created.
Process a declaration of x	Look up x. If found, compare the level of the front list item to the current level: equal -> "multiply declared variable" error; otherwise push a new item to the front of x's list. If not found, create a new one-item list.
Process a use of x	Look up x in the shared table. If not found, issue an "undeclared variable" error. No level check is needed — the front item is always the correct declaration.
On scope exit	Scan every entry; if the front item's level equals the current level, remove it (deleting the entry entirely if its list becomes empty). Decrement the current level number.

5.4 Running-Time Analysis

Operation	Running Time	Why
Scope entry	O(1)	Only a counter is incremented.
Process a declaration	O(1) expected	One hash lookup plus pushing a node to the front of a list.
Process a use	O(1) expected	One hash lookup in the single shared table.
Scope exit	O(number of names in the table)	Every entry's front item must be inspected.

5.5 Test Yourself — Full Trace

Q: Assume the symbol table is implemented using a hashtable of lists. Draw pictures to show how the symbol table changes as each declaration in the following code is processed.

```
void g(int x, int a) {
    double d;
    while (...) {
        int d, w;
        double x, b;
        if (...) {
            int a,b,c;
        }
    }
    while (...) {
        int x,y,z;
    }
}
```

Ans:

- Level 1: g: () -> void, [1] inserted.

- Level 2 (g's parameters/body): x: int, [2] and a: int, [2] inserted; then d: double, [2] inserted.
- Level 3 (first while entered): d gets int,[3] pushed to front (list: int[3] -> double[2]); w: int,[3] new; x gets double,[3] pushed to front (list: double[3] -> int[2]); b: double,[3] new.
- Level 4 (if-block entered): a gets int,[4] pushed (list: int[4] -> int[2]); b gets int,[4] pushed (list: int[4] -> double[3]); c: int,[4] new.
- Exit if-block (back to level 3): a's front item (level 4) removed, leaving int[2]; b's front item (level 4) removed, leaving double[3]; c's only item removed — c disappears entirely.
- Exit first while (back to level 2): d's front item (level 3) removed, leaving double[2]; w removed entirely; x's front item (level 3) removed, leaving int[2]; b's remaining item (level 3) removed entirely.
- Level 3 again (second while entered): x gets int,[3] pushed (list: int[3] -> int[2]); y and z are new at level 3.
- Exit second while (back to level 2): x's front item (level 3) removed, leaving int[2]; y and z removed entirely.
- Exit g's body (back to level 1): x, a, d (all at level 2) removed entirely, leaving only g in the table.

6. Comparing the Two Implementations

Aspect	List of Hashtables	Hashtable of Lists
Structure	Many small hashtables, one per open scope, linked in a list	One shared hashtable for the whole program
Scope entry cost	$O(\text{size of a new hashtable})$	$O(1)$
Declaration cost	$O(1)$ expected	$O(1)$ expected
Use (lookup) cost	$O(\text{depth of nesting})$ worst case	$O(1)$ expected
Scope exit cost	$O(1)$	$O(\text{number of names in the table})$
Best when	Uses are relatively rare, or nesting is shallow	Uses are frequent, and scope exits are relatively rare
Multiply-declared check	Look up only in the front (current) table	Look up the name, compare front item's level to current level

Q: Why might a compiler writer prefer the List-of-Hashtables method over the Hashtable-of-Lists method, or vice versa?

Ans: List of Hashtables is preferred when scope exit should be as cheap as possible (discard one table) and nesting is shallow. Hashtable of Lists is preferred when uses of names are very frequent (typical of most programs, since references vastly outnumber scope exits), because it guarantees $O(1)$ lookups regardless of nesting depth at the cost of a more expensive scope exit.

7. Type Checking

Once the symbol table is built and every use is linked to its declaration (Phase 1), the compiler performs a second traversal of the statements — type checking. Its two jobs:

- Determine the type of every expression in the program.
- Find and report type errors.

The type rules of a language specify, for every operator (including assignment), what types the operands may have and what the result type is. Both C++ and Java allow adding an int and a double, giving a double result. C++ allows assigning a double to an int (truncating); Java considers this an error.

7.1 Kinds of Type Errors Beyond Operators

Contexts that must be boolean:

- The condition of an if statement.
- The condition of a while loop.
- The termination condition of a for loop.

Errors related to method calls:

- Calling something that is not a method.
- Calling a method with the wrong number of arguments.
- Calling a method with arguments of the wrong types.

Q: List as many of the operators that can be used in a Java program as you can think of (don't forget to think about the logical and relational operators as well as the arithmetic ones). For each operator, say what types the operands may have, and what is the type of the result.

Ans:

- Arithmetic (+, -, *, /, %): numeric operands (+ also works on Strings for concatenation); result follows numeric promotion (int+int -> int, int+double -> double, any type+String -> String).
- Relational (<, >, <=, >=): numeric operands; result is boolean.
- Equality (==, !=): numeric operands, or compatible reference types (or null); result is boolean.
- Logical (&&, ||, !): boolean operands; result is boolean.
- Bitwise/shift (&, |, ^, <<, >>, >>>): integer operands (or boolean for &, |, ^); result matches the operand's integer type (or boolean).
- Assignment (=, +=, -=, *=, /=, % =, ...): right-hand side must be assignment-compatible with the left-hand side's type; result type is the left-hand side's type.
- Increment/decrement (++ , --): numeric variable operand; result type matches the variable's type.
- Conditional/ternary (cond ? a : b): cond must be boolean; result type is the common type of a and b.